

SWE 4101

Object Oriented Concepts 1

Objectives

- Understand Java program structure (basic)
- Define class and object
- Explain attributes and methods
- Understand object creation
- Explain constructors and their role

Java Features

- ➔ Object-oriented, class-based language
- ➔ Platform independent (JVM)
- ➔ Strongly typed

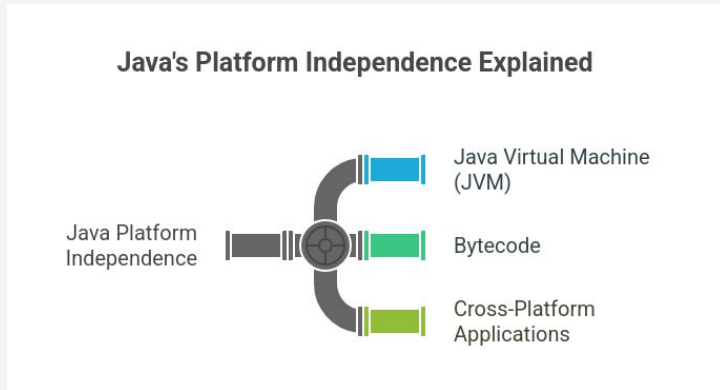
Java is Object-oriented

- ➔ It organizes software design around objects
- ➔ The language is built upon four fundamental pillars of OOP
- ➔ 99% object-oriented because of primitive data types and static keyword

Primitive data types are int, char, boolean, double, etc. These are more efficient.

The static keyword allows methods and variables to exist independently of any specific object instance.

Java is Platform-independent



- Java programs are compiled into bytecode (.class files)
- Bytecode is a universal format
- Execution is handled by the Java Virtual Machine (JVM)
- JVM acts as an interpreter
- Java libraries are built to behave consistently across platforms

“Write **once**, run **everywhere**”

Java is strongly typed

- ➔ Every variable must have a declared data type
- ➔ Data types are strictly enforced by the compiler
- ➔ Values must be compatible with the declared type
- ➔ Invalid assignments result in compile-time errors
- ➔ Type conversion must be explicit (casting) when required

Edit-Compile-Execute Cycle

Step 1. Writing the Java Program:

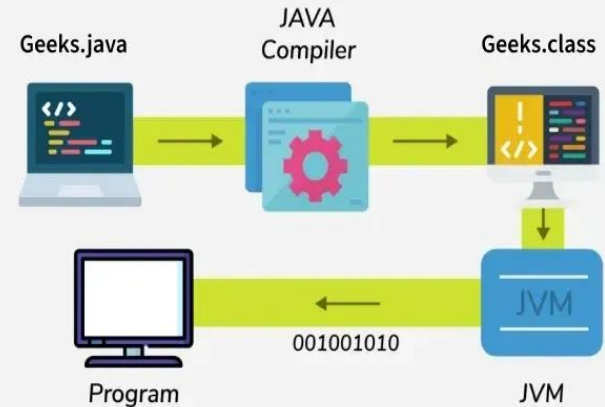
Write it in a text editor

Step 2. Compiling the Java Program:

'javac' converts the human-readable Java code into bytecode

Step 3. Executing the Java Program:

JVM interprets bytecode to machine readable code



Question

Is Java a **compiled** or an **interpreted** programming language?

It is a compiler-interpreter language

A Java Program

```
// A simple Java program
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Class name

comment

All Java programs have a main function;
acts as an entry point

Access Modifier

Function to print to screen

```
/*
This is a multi-line comment.
Everything inside these symbols is
ignored by the Java compiler.
*/
```

Another Java Program

```
public class AddNumbers {  
    public static void main(String[] args) {  
        int num1 = 5, num2 = 10, sum;  
        sum = num1 + num2;  
        System.out.println("Sum of " + num1 + " and  
            " + num2 + " is: " + sum);  
    }  
}
```

Classes (Recap)

- Real-world systems are complex
- Need a way to model entities
- Classes help organize data (attributes) and behavior (methods) together

A Java Class

```
public class Student {  
    String name;  
    int id;  
  
    void study() {  
        System.out.println("Studying...");  
    }  
}
```

Logical blueprint; does not occupy memory

Objects (Recap)

- Instance of a class
- Has:
 - ◆ Actual values (state)
 - ◆ Can perform actions (behavior)

Creating an Object

Also known as 'instantiation'

```
Student s1 = new Student();  
Student s2 = new Student();
```

new → allocates memory

s1, s2 → reference variable

Physical object; occupies memory now

Each object is independent

Accessing Attributes & Calling Methods

```
s1.name = "John Doe";  
s1.id = 101;
```

Dot operator (.) → used to assign values

```
s1.study();
```

Also known as ‘method invocation’

Question

```
public class Student {  
    String name;  
    int id;  
    double cgpa;  
    int age;  
}
```

```
Student s3 = new Student();  
s3.name = "Jane Doe";  
s3.id = 102;
```

What if we miss an attribute?

- Incomplete object
- Error-prone

Introducing: Constructor

A constructor is a special method that is automatically called when an object is created.

- Ensure object is initialized properly
- Enforce required data
- Same name as the class
- Automatically invoked
- No return type (not even void)

Constructor (Continued)

```
class Student {  
    String name;  
    int id;  
    double cgpa;  
    int age;  
  
    Student(String name, int id, double cgpa, int age) {  
        this.name = name;  
        this.id = id;  
        this.cgpa = cgpa;  
        this.age = age;  
    }  
}
```

Constructor (Continued)

```
Student s4 = new Student("Bob", 101, 3.50, 20);
```

Thank You